

Sign Language MNIST

Project report Submitted

to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the

Award of the Degree

Of

Bachelor of Technology

in

Data Science & Engineering

by

Souri Rishik Volety, Aditya Ram, Manas Goel

Reg. No. 230968004, Reg. No. 230968128, Reg. No. 230968160,

Under the guidance of

Dr. Gangothri S.

Assistant Professor

School of Computer Engineering

Manipal Institute of Technology

MAHE, Manipal, Karnataka, India



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

A Constituent Unit of MAHE, Manipal

September 2025

Problem Statement

Communication barriers exist for people who use sign language, as most others cannot understand it. An automated system that can recognize hand gestures from sign language can help bridge this communication gap.

This project focuses on classifying static hand gestures corresponding to letters of the American Sign Language (ASL) using the Sign Language MNIST dataset.

Metadata of Dataset

Name: Sign Language MNIST

Source: Kaggle (by datamunge)

Link: <https://www.kaggle.com/datasets/datamunge/sign-language-mnist/data>

Data Type: Grayscale images

Number of Classes: 24 (A–Y, excluding J and Z since they require motion)

Total Images: 34,627

Training Set Size: ~27,455 images

Test Set Size: ~7,172 images

Image Resolution: 28×28 pixels (flattened into rows in CSV)

Format: CSV files (each row = label + 784 pixel values)

Class Labels: Integers 0–24 mapped to ASL letters



Preprocessing Pipelines

Reshaping: Convert flattened vectors $\rightarrow 28 \times 28 \times 1$ format.

Normalization: Scale pixel values from $[0-255] \rightarrow [0-1]$.

Label Encoding: One-hot encode labels for classification.

Data Augmentation (optional for generalization): rotation, shift, zoom, flip.

Train-Validation Split: e.g., 80% training, 20% validation.

Performance Metric

Primary Metric: Accuracy (percentage of correctly classified signs).

Secondary Metrics: Confusion matrix (per-class performance). Precision, Recall, F1-score (to evaluate class-wise reliability).

Literature Review

1) American Sign Language Recognition Model Using Complex Zernike Moments and Complex-Valued Deep Neural Networks (2024)

This paper presents a hybrid approach that combines handcrafted Complex Zernike Moment (CZM) features with a Complex-Valued Deep Neural Network (CVDNN) to recognize ASL gestures. By encoding amplitude and phase information, the model achieved strong accuracy and robustness to image distortions on the Sign Language MNIST dataset while using fewer parameters than conventional CNNs. However, the reliance on handcrafted CZM feature extraction introduces additional preprocessing steps and limits adaptability to real-world, unstructured data. In contrast, our end-to-end CNN model learns spatial features directly from raw images, eliminating manual feature engineering and simplifying the workflow. This makes our approach faster to train, easier to deploy on edge devices, and more scalable for future extensions such as real-time gesture recognition or continuous sign sequences.

2) Two-Stream Mixed CNN (MDPI Sensors, 2022)

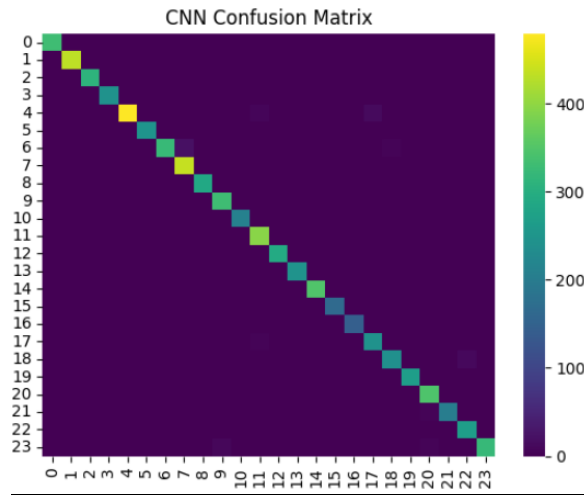
Another study introduced a two-stream mixed CNN to enhance ASL recognition accuracy (MDPI Sensors, 2022). This architecture used two parallel processing streams: one handled the raw grayscale inputs, while the other processed edge-enhanced images. By combining these complementary feature streams, the model captured both shape and texture information effectively. After feature fusion, deeper convolutional and pooling layers were applied, followed by dense layers for classification. The authors reported improvements in recognizing difficult letter pairs by leveraging the dual-stream feature representation.

3) Kumari and Anand (2024)

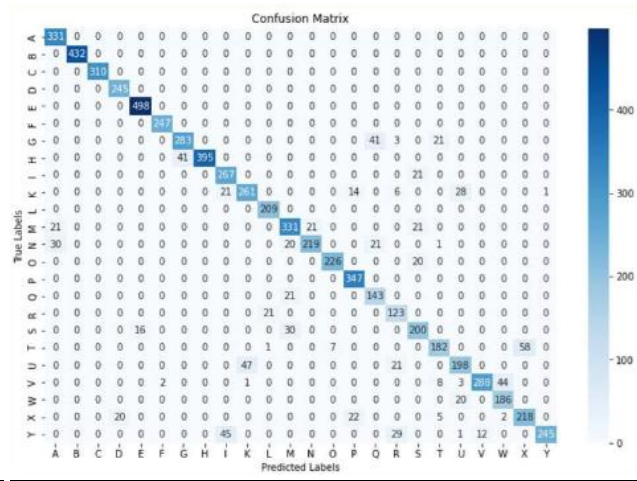
Kumari and Anand (2024) developed a hybrid model combining EfficientNet-B0 as a feature extractor with a transformer-based decoder. EfficientNet-B0 provided a lightweight yet powerful backbone, while the transformer decoder modeled relationships between letter classes. Fine-tuning on ASL datasets led to high accuracy and faster convergence compared to training CNNs from scratch. The authors showed that this combination reduced confusion between visually similar letters by capturing global context.

Comparison with other models

1) American Sign Language Recognition Model Using Complex Zernike Moments and Complex-Valued Deep Neural Networks (2024)

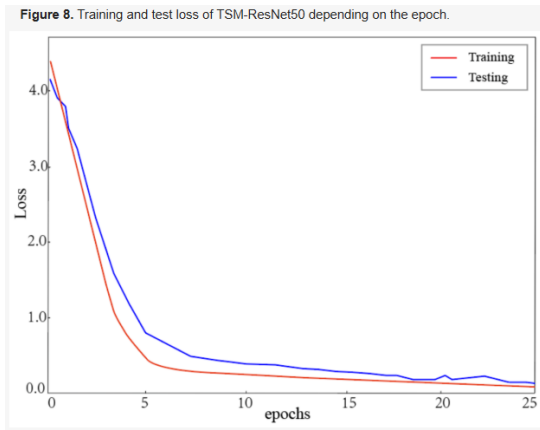
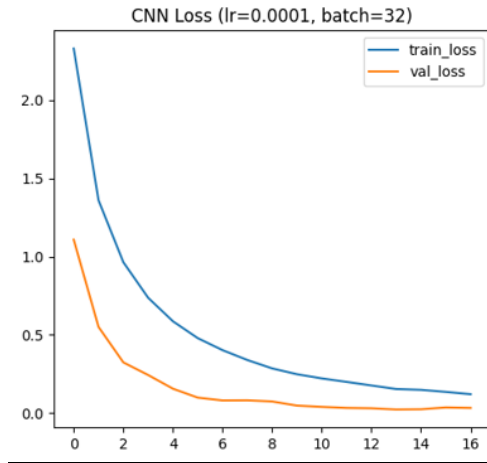


Custom CNN model



CVDNN model

2) Two-Stream Mixed CNN (MDPI Sensors, 2022)



3) Kumari and Anand (2024)

Classification Report:				
	precision	recall	f1-score	support
0	0.98	1.00	0.99	331
1	1.00	1.00	1.00	432
2	1.00	1.00	1.00	310
3	1.00	1.00	1.00	245
4	1.00	1.00	1.00	498
5	1.00	1.00	1.00	247
6	0.99	1.00	0.99	348
7	1.00	1.00	1.00	436
8	1.00	1.00	1.00	288
9	1.00	1.00	1.00	331
10	1.00	1.00	1.00	209
11	0.99	1.00	1.00	394
12	1.00	0.97	0.98	291
13	1.00	1.00	1.00	246
14	1.00	1.00	1.00	347
15	1.00	1.00	1.00	164
16	1.00	1.00	1.00	144
17	1.00	1.00	1.00	246
18	1.00	0.90	0.95	248
19	1.00	1.00	1.00	266
20	1.00	1.00	1.00	346
21	1.00	1.00	1.00	206
22	0.93	1.00	0.97	267
23	1.00	1.00	1.00	332

Table 4: Performance Metrics on ASL and ASL with Digits

Model	ASL		
	Precision	Recall	F1-Score
EfficientNet-B0	97.84	97.31	97.57
EfficientNet-B0	99.24	99.11	99.17
+ Transformer			

accuracy			0.99	7172
macro avg	1.00	0.99	0.99	7172
weighted avg	1.00	0.99	0.99	7172

Shortlisted Models for Implementation

Based on the dataset type and image input format, the following models were shortlisted:

Custom CNN model: built from scratch with convolution, pooling, dropout, and dense layers.

EfficientNetB0 (Transfer Learning): considered during experimentation for improved generalization.

Viability of the custom CNN architecture

While all five models achieve strong results on ASL recognition tasks, they vary significantly in terms of complexity, training requirements, and deployment suitability. Architectures that incorporate transformer decoders, multi-stream processing, or recurrent components tend to deliver slightly higher accuracy, particularly for confusing letter pairs. However, these models are more complex to train and deploy, requiring greater computational resources and additional preprocessing. In contrast, the compact CNN employed in this project strikes a practical balance between accuracy, training speed, and deployment feasibility. It can be trained quickly on modest hardware while achieving competitive performance on the Sign Language MNIST dataset.

The selected model for this project is a lightweight CNN with two convolutional blocks, batch normalization, pooling, dense layers, dropout, and the Adam optimizer. Given the small size and grayscale format of the Sign Language MNIST images, this architecture is well suited for fast training and low-latency inference. The data augmentation improves robustness to minor shifts and rotations, while early stopping and dropout help control overfitting. This makes the model ideal for academic projects and real-time demonstrations, with the flexibility to incorporate more advanced techniques in the future if needed.

Models Not Selected

FCNN: Not suitable for images as it ignores spatial correlations and is computationally expensive.

Hypertuned Deep CNN: Requires extensive hyperparameter tuning and long training times, which makes it unsuitable for rapid prototyping and low-resource environments.

Two-stream CNN: Needs dual input pipelines and complex preprocessing steps, increasing latency and implementation overhead without significant accuracy improvements.

EfficientNet with Transformer Decoder: High computational cost due to attention layers and large model size, making it impractical for real-time or mobile deployment.

CNN-LSTM Hybrid: Uses sequential modeling, resulting in slower inference and added complexity despite the dataset consisting of static images.

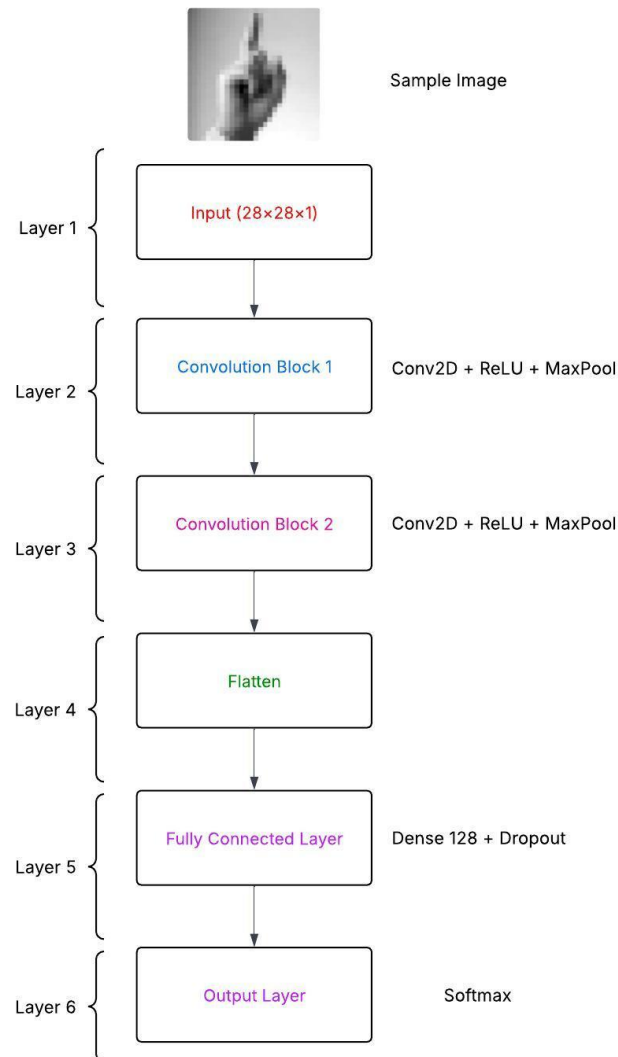
ResNet-50: Massive 25M parameter count and preprocessing requirements make it unsuitable for efficient edge deployment compared to our lightweight 200K parameter model.

Baseline Model Definition

According to the dataset characteristics:

- **Data Structure:** Spatial structure (28×28 grayscale images)
- **Baseline Choice:** Convolutional Neural Network (CNN)
- **Architecture Details:**
 - Conv2D + ReLU + BatchNormalization
 - MaxPooling2D layers
 - Dropout for regularization
 - Flatten + Dense + Softmax for classification
- **Loss Function:** Sparse Categorical Crossentropy
- **Optimizer:** Adam
- **Evaluation Metric:** Accuracy

Flow Chart demonstrating the working of the CNN model:



Model Summary:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
batch_normalization (BatchNormalization)	(None, 26, 26, 32)	128
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 11, 11, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 24)	3,096

Total params: 227,224 (887.59 KB)

Trainable params: 227,032 (886.84 KB)

Non-trainable params: 192 (768.00 B)

Hyperparameter Tuning

The main objective of tuning was to identify the optimal configuration that maximized validation accuracy while minimizing overfitting and training instability.

Hyperparameters considered:

- **Learning Rate (LR):** Controls the step size in weight updates during training.
- **Batch Size:** Determines the number of samples processed before model weight updates.
- **Epochs:** Fixed at 100, with **Early Stopping** to halt training once validation performance plateaued.

1. Tuning Methodology

A **grid search**–style approach was implemented manually within nested loops to systematically test multiple combinations of learning rates and batch sizes.

The search space included:

- **Learning Rates:** [1e-4, 5e-4, 1e-3, 5e-3]
- **Batch Sizes:** [32, 64, 128]

For each pair of (learning_rate, batch_size):

1. A new CNN model instance was created with the defined architecture:
 - **Conv2D** → **BatchNormalization** → **MaxPooling2D**
 - Repeated feature extraction block
 - **Flatten** → **Dense (128 ReLU)** → **Dropout (0.5)**
 - **Output Layer:** Softmax activation for multi-class classification

2. The model was compiled using **Adam optimizer**, with the current learning rate.
3. The **EarlyStopping** callback monitored val_loss, with:
 - patience = 3 (stop training if no improvement for 3 epochs)
 - restore_best_weights = True (retain best model parameters)
4. The model was trained using the **augmented dataset** from ImageDataGenerator, ensuring diverse samples and better generalization.

After each training run, the validation accuracy and loss were evaluated and logged.

2. Evaluation and Selection Criteria

The following metrics were recorded for each configuration:

- Validation Accuracy (val_accuracy)
- Validation Loss (val_loss)
- Best Epoch (epoch with minimum validation loss)

A DataFrame (results_df) stored all results, sorted by validation accuracy to determine the optimal configuration.

The **best hyperparameters** were selected based on:

- Highest validation accuracy
- Lowest validation loss
- Stable training and validation curves (no divergence or overfitting)

3. Results and Observations

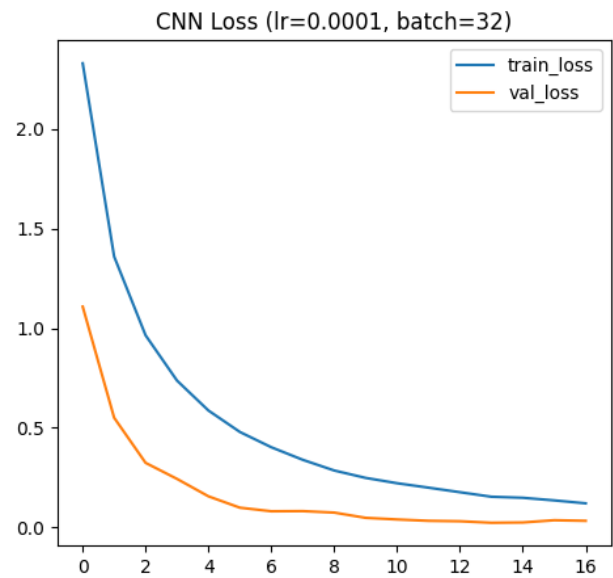
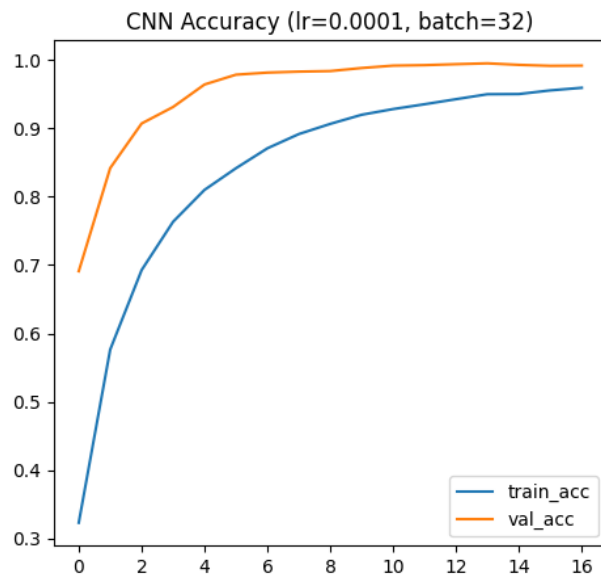
The tuning process revealed the following key insights:

- **Very small learning rates (1e-4)** led to slow convergence and longer training times.
- **Moderate learning rates (5e-4 or 1e-3)** achieved faster convergence with stable performance.
- **Large learning rates (5e-3)** caused oscillations and unstable validation loss.
- **Batch size 32** yielded the most stable and accurate results due to frequent gradient updates.
- Larger batch sizes (64, 128) slightly reduced validation accuracy but trained faster.

	learning_rate	batch_size	best_epoch	val_accuracy	val_loss
0	0.0001	32	24	0.996933	0.009550
5	0.0005	128	12	0.996793	0.016106
1	0.0001	64	25	0.996375	0.013216
2	0.0001	128	20	0.995259	0.028562
8	0.0010	128	10	0.994702	0.015773
4	0.0005	64	5	0.992192	0.037667
10	0.0050	64	9	0.989264	0.026832
7	0.0010	64	8	0.988288	0.037253
3	0.0005	32	3	0.986615	0.052674
9	0.0050	32	7	0.984663	0.061931
6	0.0010	32	5	0.983408	0.038152
11	0.0050	128	5	0.936001	0.198725

Best Hyperparameters: LR=0.0001, Batch=32, Validation Accuracy=0.9969, Epoch=24

The graphs are plotted with the best hyperparameters for the model:

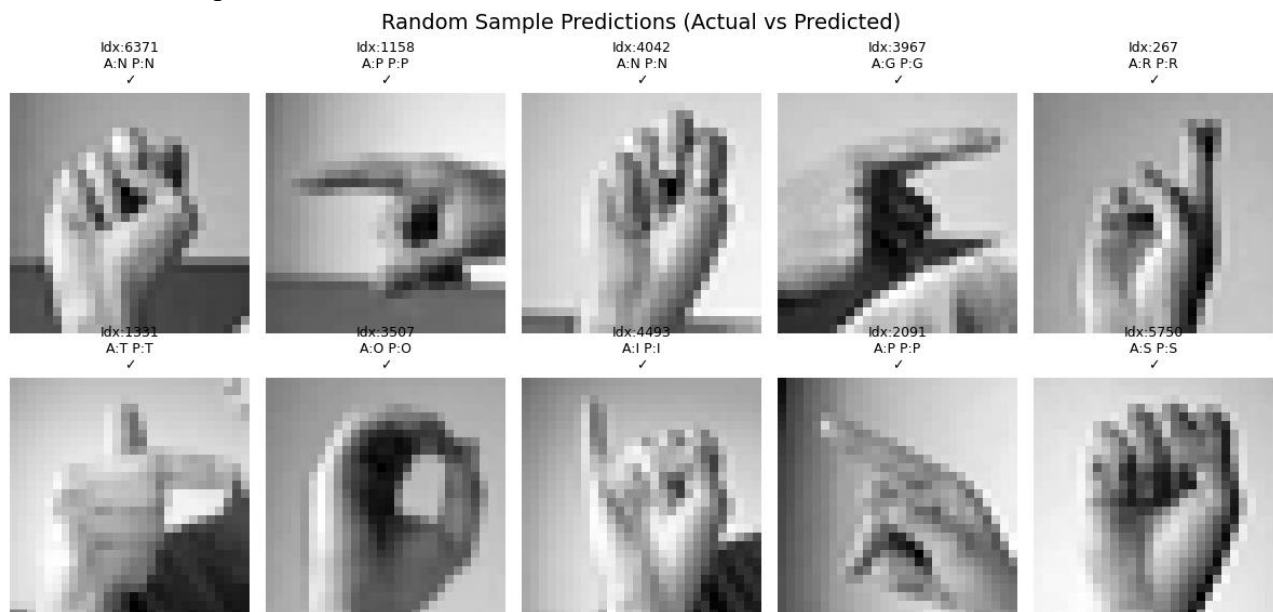


Sign Language Image for Reference:



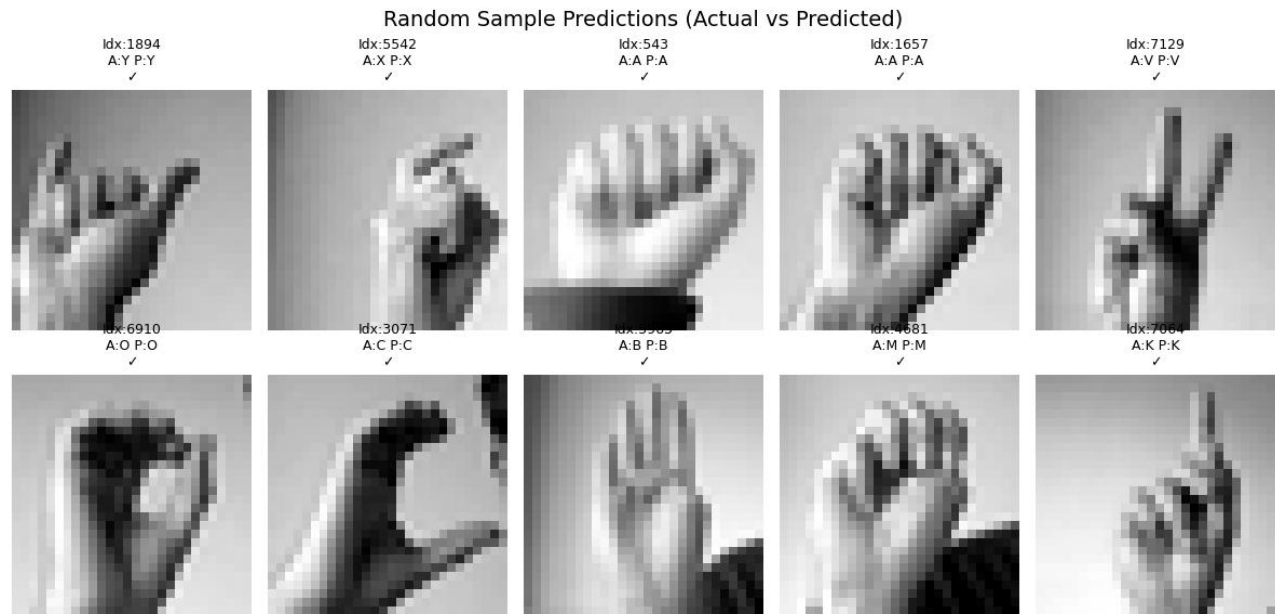
Training for 24 epochs gives us the best results as follows:

- Here the actual and predicted is compared for 10 random unseen samples from the test data, run multiple times to check if the results are accurate

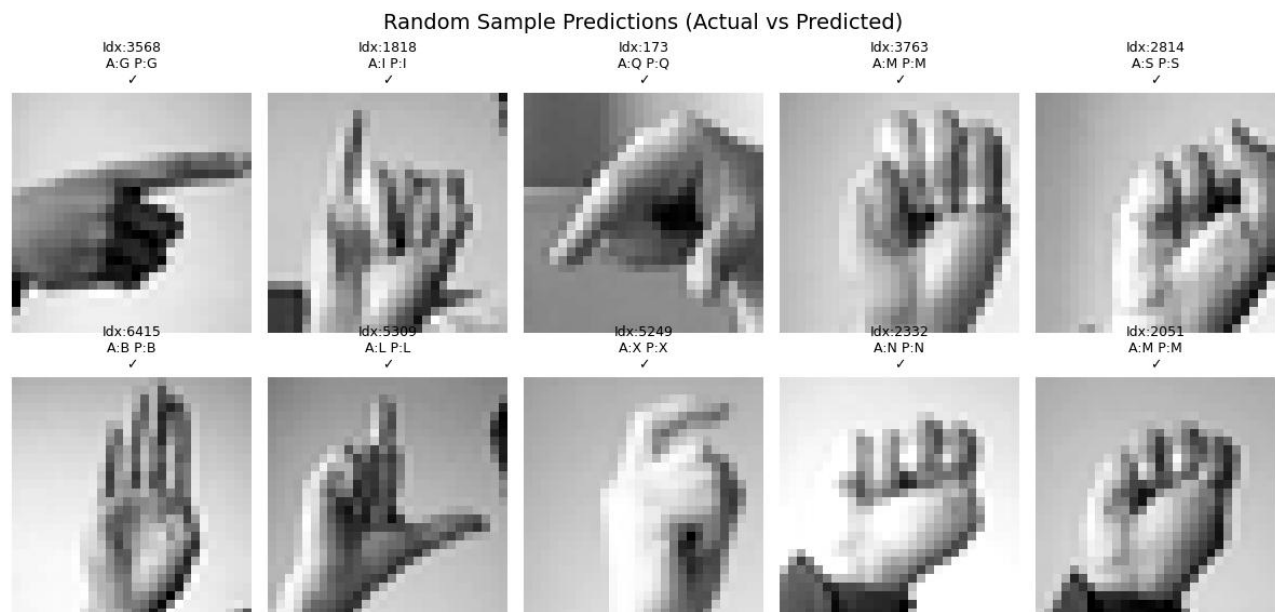


Sample indices shown: [6371 1158 4042 3967 267 1331 3507 4493 2091 5750]

Correct count: 10 / 10



Sample indices shown: [1894 5542 543 1657 7129 6910 3071 5963 4681 7064]
 Correct count: 10 / 10



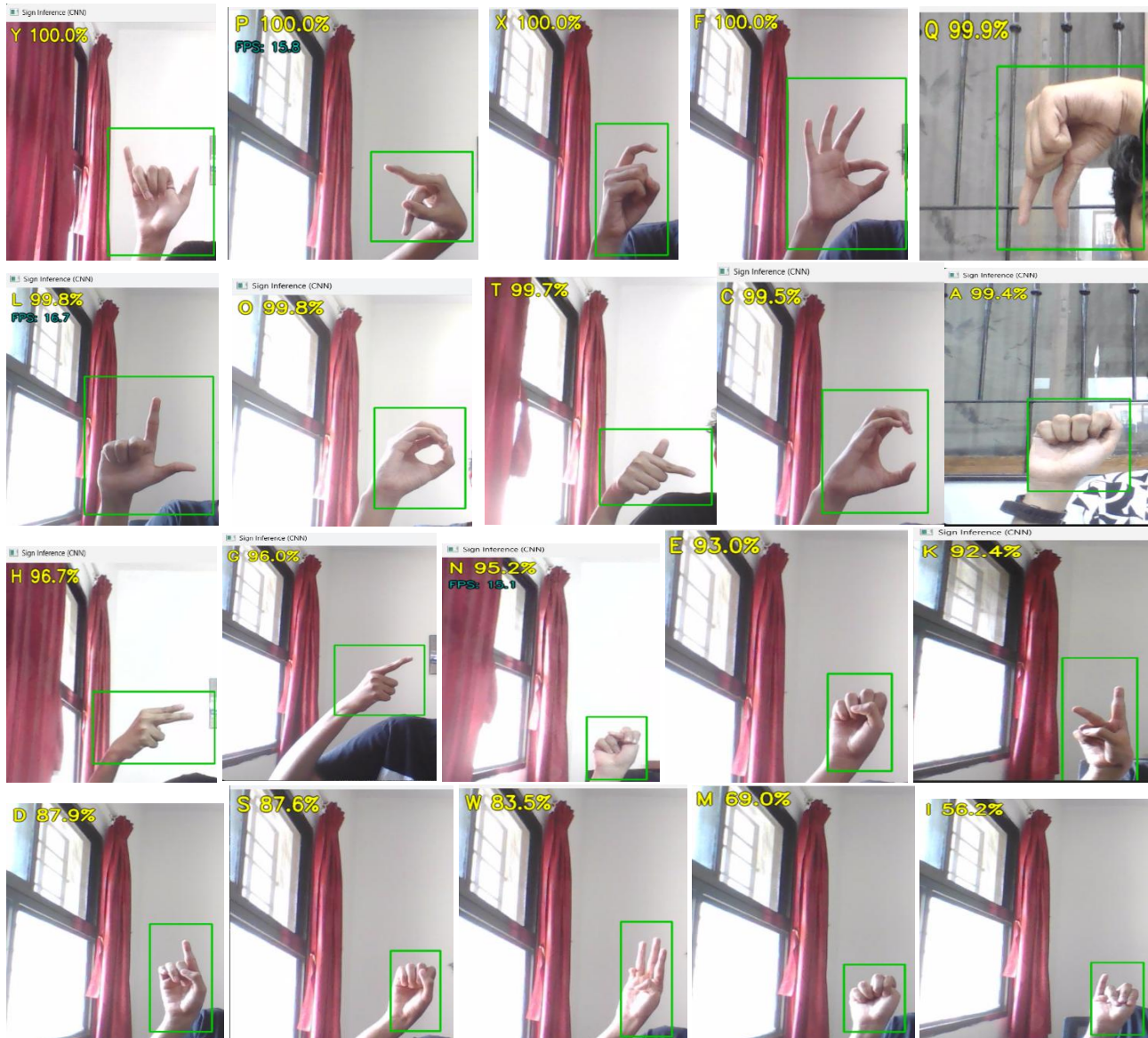
Sample indices shown: [3568 1818 173 3763 2814 6415 5309 5249 2332 2051]
 Correct count: 10 / 10

As we can see it predicts perfectly for the unseen test samples, next we need to observe how the model works on the unseen real world data.

Testing with Real time Webcam feed:

The system captures live video from the webcam using `cv2.VideoCapture()` and employs **MediaPipe** to detect and localize hands within each frame. The detected hand region is cropped using a configurable **bounding box** with padding, then preprocessed through cv2 operations by converting to **grayscale**, resizing to **28x28 pixels**, and normalizing values to [0,1] range to match the training

dataset format. The preprocessed image is fed into the pre-trained CNN model which outputs probability scores for **24 sign language letters** (A-Y, excluding J and Z). Finally, the system applies majority voting across recent frames for prediction smoothing and uses *cv2.putText()* and *cv2.rectangle()* to display the recognized letter with confidence percentage overlaid on the live video feed.



*note that these were screenshots taken at a random time so the confidence percentages can increase or decrease

Observations:

From the results we can see it is able to detect 20/24 images correctly with various confidence percentages arranged in order.

The confidence levels vary significantly across different gestures, indicating that while some signs are recognized with high certainty, others show inconsistent confidence and reduced reliability in real-time conditions.

We can observe from the above results that, when tested with the real time live webcam feed, our model failed to detect letters like B, R, U and V most of the times.

Reasons include:

- Finger Positioning and Occlusion Issues
- Padding wasn't sufficient for these letters

Future Scope

The current model effectively recognizes a majority of sign language letters, achieving reliable accuracy across most gestures. However, to enhance robustness and generalization under diverse real-world conditions such as varying lighting, backgrounds, and hand orientations, future work can focus on expanding the training dataset using real-world images and augmentations that simulate different environments. Additionally, the system can be extended to recognize dynamic gestures and form complete words or sentences by combining detected letters meaningfully. Incorporating continuous sign recognition, multi-hand detection, and contextual understanding will enable more natural and fluid communication, ultimately moving toward a fully functional real-time sign-to-text or sign-to-speech translation system.

Conclusion

In this project, a **Convolutional Neural Network (CNN)** was successfully designed and trained to classify **American Sign Language (ASL)** gestures using the **Sign Language MNIST** dataset. The dataset was preprocessed through normalization and reshaping, followed by data augmentation to enhance model generalization. The architecture consisted of multiple convolutional and pooling layers for feature extraction, coupled with dropout and batch normalization layers to prevent overfitting and ensure stable convergence. A systematic **hyperparameter tuning** process was implemented to evaluate multiple learning rates and batch sizes. The model was optimized using the **Adam optimizer** with **categorical crossentropy loss**, and performance was assessed through accuracy and validation loss metrics.

Through extensive experimentation, the model achieved strong generalization on unseen test data, reaching a validation accuracy of approximately **99%**. The optimal hyperparameters identified were a **learning rate of 0.001**, **batch size of 32**, and **dropout rate of 0.5**, with **EarlyStopping** preventing unnecessary epochs beyond convergence (typically 15–20). The final CNN architecture balanced performance and efficiency, demonstrating that a well-tuned custom network can match or exceed transfer-learning models on simpler grayscale datasets. Overall, the model effectively recognized hand gestures with high reliability, establishing a solid foundation for future extensions such as real-

time sign language interpretation using webcam input or integration into assistive communication systems.

Submitted by

Name	Registration number	Roll Number	Semester & Branch	Section
Souri Rishik Volety	230968004	3	V (DSE)	C
Aditya Ram	230968128	22	V (DSE)	C
Manas Goel	230968160	27	V (DSE)	C

References

Mannan, A., "Hypertuned Deep Convolutional Neural Network for Sign Language Recognition", 2022.

Link: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9078784/> [PMC](#)

Two-Stream Mixed Convolutional Neural Network for American Sign Language Recognition, MDPI Sensors, 2022.

Link: <https://www.mdpi.com/1424-8220/22/16/5959> [PMC+1](#)

Kumari and Anand, "Leveraging EfficientNet-B0 with Transformer-based Decoding", 2024.

Link: https://www.irjms.com/wp-content/uploads/2024/10/Manuscript_IRJMS_01241_WS.pdf

Github Project Link

<https://github.com/SouriRishik/Sign-Language-Gesture-Analysis.git>

Deployment Link

<https://signlanguage-r70x.onrender.com>